

Homework 1: Designing Context-Free Grammars

Haotian Li, Department of Neuroscience

1.4 Questions

1.4.1 Provide 10 random sentences generated from your script.

- (1) every floor understood a perplexed president !
- (2) every sandwich on every sandwich wanted the president with every chief of staff with the pickle !
- (3) the pickled pickle understood a fine floor .
- (4) a chief of staff understood a president .
- (5) is it true that a pickle under the president kissed every delicious president on a fine delicious chief of staff on a perplexed floor ?
- (6) is it true that a floor wanted a perplexed pickle ?
- (7) every pickle wanted the president with the fine president under every chief of staff in a sandwich under a floor with the chief of staff with every chief of staff on the chief of staff under every pickled sandwich on every floor in a president with every floor on a sandwich under the chief of staff on a chief of staff in every president under the floor with a pickled floor in the president under the perplexed pickle on the president on the pickle on every president in every floor in every chief of staff under the sandwich in the president on every pickle in the perplexed floor with a perplexed sandwich in a pickle with a delicious perplexed chief of staff in a fine sandwich under every pickle under the chief of staff under every perplexed floor with every fine pickle with every floor in the chief of staff on a pickled sandwich on a chief of staff on every chief of staff in a floor in every perplexed sandwich on every pickle under the sandwich with the floor with every floor under every perplexed chief of staff on every chief of staff on the floor in a sandwich .
- (8) the perplexed president ate every sandwich under the floor !
- (9) the floor kissed the pickle .
- (10) the chief of staff on a chief of staff in the chief of staff on a pickle in the perplexed pickle with the pickle under the pickle understood a president .

1.4.2 Provide 2 random sentences generated from your script, using --tree to show their derivations.

(1)

(ROOT (is it

true

that

(S ((NP ((NP ((Det (every))

(Noun ((Adj (fine))

(Noun (chief of

staff))))))

```

      (PP ((Prep (in ))
        (NP ((NP ((Det (the ))
          (Noun (pickle ))))
          (PP ((Prep (with ))
            (NP ((Det (every ))
              (Noun ((Adj (pickled ))
                (Noun ((Adj (fine ))
                  (Noun (president ))))))))))))))))
    (VP ((Verb (ate ))
      (NP ((NP ((NP ((Det (every ))
        (Noun (pickle ))))
        (PP ((Prep (under ))
          (NP ((Det (every ))
            (Noun (floor ))))))))
      (PP ((Prep (with ))
        (NP ((NP ((Det (a ))
          (Noun (floor ))))
          (PP ((Prep (in ))
            (NP ((NP ((NP ((Det (the ))
              (Noun ((Adj (fine ))
                (Noun (sandwich ))))))
              (PP ((Prep (in ))
                (NP ((Det (a ))
                  (Noun (floor ))))))))
              (PP ((Prep (in ))
                (NP ((Det (every ))
                  (Noun (president ))))))))))))))))
      ?))
  (2)
  (ROOT ((S ((NP ((Det (a ))
    (Noun ((Adj (perplexed ))
      (Noun (chief of

```

```

staff))))))
(VP ((Verb (ate ))
  (NP ((NP ((Det (a ))
    (Noun (president ))))
    (PP ((Prep (in ))
      (NP ((Det (the ))
        (Noun (floor ))))))))))))
!))

```

1.4.3 As in the previous question, but with a --max expansions of 5.

```

(1) (ROOT (is it
  true
  that
  (S ((NP ((Det (a ))
    (... (president ))))
    ...))
  ...))
(2) (ROOT ((S ((NP ((Det (the ))
  (... (Adj Noun))))
  ...))
  ...))

```

2.1 Questions

1. Why does your program generate so many long sentences? Specifically, what grammar rule (or rules) is (or are) responsible and why? What is special about it/them?

There are three rules should be responsible for long sentences: "VP -> Verb NP", "NP -> NP PP" and "PP -> Prep NP". Among them, "NP -> NP PP" has the strongest power to elongate the sentences.

For "VP -> Verb NP" and "PP -> Prep NP", they recruit a new word and turn the phrase into NP. For "NP -> NP PP", because PP will turns into Prep NP, it basically doubles the length of the sentence ("NP -> NP Prep NP",). Therefore, if "NP -> NP PP" happens many times the sentence will become extremely long.

2. The grammar allows multiple adjectives, as in the fine perplexed pickle. Why do your program's sentences do this so rarely? (Give a simple mathematical argument.)

Because the decision from "Noun" to its downstream words has multiple choices. There is only a one-in-six chance that "Noun -> Adj Noun" occurs. In other cases, "Noun" will turn into a terminal word like "president" or "sandwich". If a noun has multiple adjectives ahead of it, it means the "Noun -> Adj Noun" process has occurred multiple times, which has the probability $(1/6)^n$. And that is the reason why n wouldn't be so large.

3. Which numbers must you modify to fix the problems in item 1 and item 2, making the sentences shorter and the adjectives more frequent? Put these adjustments in a new grammar file named grammar2.gr. Check your answer by running your generator!

We can just decrease the weight of "NP -> NP PP" to 0.1 and increase the weight of "Noun -> Adj Noun" to 5. The following content generated by my generator validates my modification:

(1) a delicious fine floor wanted a delicious fine delicious delicious delicious pickled delicious pickled chief of staff under the pickled perplexed perplexed chief of staff .

(2) a perplexed president ate a perplexed pickled president .

4. What other numeric adjustments can you make to grammar2.gr in order to favor more natural sets of sentences? Experiment. Explain the changes.

I added more Verb, Det, Noun, Adj, Prep options. I also added more sentence structures (clauses) to make the sentences generated more complicated. The sentences generated by grammar2.gr are demonstrated below.

5. Provide 10 random sentences generated with the grammar2.gr.

(1) do you think that each sandwich ate every table ?

(2) did you know that the chef pickled the pickled chef ?

(3) is it true that a chief of staff dropped a mysterious president ?

(4) do you think that each president in each sandwich beside the delicious enormous senator kissed that senator ?

(5) this pickle ate every floor !

(6) is it true that each sandwich found every chef ?

- (7) is it true that each fine green chef dropped every senator ?
- (8) do you think that this table dropped that fine angry chef beside a president ?
- (9) did you know that this perplexed senator pickled every green sandwich ?
- (10) do you think that every delicious table wanted every table ?

2.3 Questions continued

9. Briefly discuss your modifications to the grammar.

I made 3 major modifications to the grammar file:

(1) I differentiated transitive verbs and intransitive verbs using the rule:

1 VP Verb_trans NP

1 VP Verb_intrans

(2) I added conjunction words “and” and “or” using rules:

1 NP NP_Conj Conj NP_Conj

1 NP_Conj NP_Conj PP

Note that since a phrase can't have multiple conjunction words like “A and B and C”, I differentiated NP and NP_Conj using the subscript “_Conj”. In this way the multiple conjunction words' pitfall was avoided.

(3) I added the sentence pattern:

1 ROOT it VP that S .

10. Provide 10 random sentences generated with grammar3.gr that illustrate your modifications.

(1) do you think that each perplexed castle wanted the pickled table and the perplexed president ?

(2) a coconut and every chef ran !

(3) that coconut and every president or the senator cried .

(4) that angry pickle and every castle in that pickle or each table and each table and this coconut and the senator or every castle on this chief of staff near this castle or this chef beside each coconut under every table with that mysterious sandwich in the coconut in every perplexed table and the coconut behind this castle beside this senator or every enormous pickle near a president or every delicious pickle above the chief of staff or the pickled pickle near this mysterious coconut and this chef in this chef and the sandwich on the pickle and a pickled delicious sandwich in the enormous sandwich under a table in each sandwich and every sandwich and that sandwich under each fine sandwich beside every sandwich and this delicious chef and a mysterious sandwich and that president or every pickled pickle behind each president near each president or this table behind this chief of staff beside a floor beside each table above a president under that perplexed table and this coconut or this sandwich on a pickle and this table on that floor and the castle with every green angry pickled floor on every delicious sandwich and a

(5) did you know that a castle ran ?

PP NP_Conj ... NP_Conj NP_Conj NP_Conj ...
NP_Conj ... NP_Conj NP_Conj ... NP_Conj NP_Conj ...
NP_Conj ... NP_Conj NP_Conj ... NP_Conj ... NP_Conj ...
NP_Conj NP_Conj ... NP_Conj NP_Conj ... NP_Conj ...
NP_Conj NP_Conj ... NP_Conj

NP_Conj ... NP_Conj ... NP_Conj' ... NP_Conj' ... NP_Conj ...
 NP_Conj ... NP_Conj ... NP_Conj ... NP_Conj ... NP_Conj ...
 NP_Conj ...

(8) that senator carried a president and a sandwich and that pickle behind each perplexed perplexed floor beside this castle in the delicious perplexed senator and each enormous senator and the angry pickle near that delicious senator on that mysterious pickle under this angry green chief of staff and a table or each chef beside each delicious table or that mysterious pickled president under this floor or every fine floor and each chief of staff and each angry president near that chief of staff or a table under

(a) One derivation is as follows; what is the other?

The other way of derivation could be written as:

```
(ROOT (S (NP (NP (Det every)
                (Noun sandwich))
            (PP (Prep with)
                (NP (NP (Det a)
                    (Noun pickle))
                    (PP (Prep on)
                        (NP (Det the)
                            (Noun floor))))))
            (VP (Verb wanted)
                (NP (Det a)
                    (Noun President))))
    .)
```

(b) Is there any reason to care which derivation was used? Consider meaning.

Yes. In the first case, the “pickle” in the sandwich is a regular pickle. But in the second case, the “pickle” is a pickle lying on the floor (though I have no idea what kind of pickle it is). There is ambiguity.

3.3 Questions continued

2. Sentences generated by grammar2.gr and parser are printed below (for each sentence, the first tree is generated by grammar file and the second tree is reconstructed by parse):

(1)

```
(ROOT (is it
      true
      that
      (S ((NP ((Det (this ))
                (Noun ((Adj (mysterious ))
                    (Noun (coconut ))))))
          (VP ((Verb (found ))
              (NP ((Det (a ))
                  (Noun ((Adj (angry ))
                      (Noun ((Adj (delicious ))
                          (Noun (table ))))))))))))
```

?)

(ROOT is it true that (S (NP (Det this) (Noun (Adj mysterious) (Noun coconut))) (VP (Verb found) (NP (Det a) (Noun (Adj angry) (Noun (Adj delicious) (Noun table)))))) ?)

This one looks fine.

(2)

(ROOT ((S ((NP ((Det (each))
 (Noun ((Adj (pickled))
 (Noun (chief of
 staff)))))
 (VP ((Verb (ate))
 (NP ((Det (that))
 (Noun (coconut)))))))))
 .))

(ROOT (S (NP (Det each) (Noun (Adj pickled) (Noun chief of staff))) (VP (Verb ate) (NP (Det that) (Noun coconut)))) .)

This one looks fine.

(3)

(ROOT ((S ((NP ((Det (this))
 (Noun (floor))))
 (VP ((Verb (carried))
 (NP ((Det (each))
 (Noun ((Adj (pickled))
 (Noun (coconut)))))))))
 !))

(ROOT (S (NP (Det this) (Noun floor)) (VP (Verb carried) (NP (Det each) (Noun (Adj pickled) (Noun coconut)))) .)

This one looks fine.

(4)

(ROOT (do you
 think
 that

```

(S ((NP ((Det (the ))
        (Noun (pickle ))))
  (VP ((Verb (admired ))
        (NP ((Det (that ))
              (Noun (pickle ))))))))
  ?))

```

(ROOT do you think that (S (NP (Det the) (Noun pickle)) (VP (Verb admired) (NP (Det that) (Noun pickle)))) ?)

This one looks fine.

```

(5)
(ROOT ((S ((NP ((Det (that ))
                (Noun (chef ))))
  (VP ((Verb (dropped ))
        (NP ((Det (this ))
              (Noun ((Adj (green ))
                    (Noun (chief of
                          staff))))))))))
  .))

```

(ROOT (S (NP (Det that) (Noun chef)) (VP (Verb dropped) (NP (Det this) (Noun (Adj green) (Noun chief of staff)))) .)

This one looks fine.

All sentences (at least in my experiment) generated by grammar2.gr are reconstructed correctly (same tree structure) by parse. But I believe that there are cases where parse produces different tree structures. Maybe my grammar is not complicated enough to make that.

3. There are five possible ways to parse the sentence. Let's forget about the "grammar tree", the sentence is basically 4 Nouns connected by Preps. How many ways to interpret a sentence like this? It became the bracket problem we solved in recitation class. $f(4)=f(1)f(3)+f(2)f(2)+f(3)f(1)=2+1+2=5$. The parse program also claims that there are 5 possibilities.

4. Examples generated using grammar.gr:

(1) every pickle with every chief of staff on every pickle ate the floor under every chief of staff !

number of parses = 2

(2) is it true that the chief of staff with a floor under every president under every fine president in the delicious pickle wanted every chief of staff ?

number of parses = 14

(3) a floor on the floor under every sandwich in a fine pickle with every president on every delicious sandwich with every president on the floor under the pickle in a sandwich under a sandwich under every sandwich in a pickle in the sandwich ate every floor under the president in every pickle with the floor in the president with every chief of staff on the fine chief of staff under a chief of staff with every floor with a chief of staff with a perplexed pickle on every chief of staff .

number of parses = 43672119400

Of course, longer sentences have larger parse numbers. Just like $f(n)$ grows as n increases.

5.

(a) $5.144e-05$ comes from $1/(3*2*3*6*5*2*3*6)$. Just multiply the probability at each crossing. $P(\text{sentence})$ is equal to $P(\text{best_parse})$ because there is only one possible parse. $P(\text{best_parse}|\text{sentence})=1$ is also because there is only one possible parse.

(b) $P(\text{best_parse}|\text{sentence}) = P(\text{best_parse}, \text{sentence}) / P(\text{sentence}) = 6.202 / 12.40 = 0.50016$, not exactly 0.5.

(c) $H(P, Q) = -\sum_i p_i \log(q_i)$
 $= -p(\text{'the'}) \log(q(\text{'the'})) - p(\text{'president'}) \log(q(\text{'president'})) - \dots$

Where $p(\text{'the'}) = p(\text{'president'}) = \dots = p(\text{'.'}) = 1/18$ since p is the sampled probability, not the bulk probability. Therefore,

$H(P, Q) = -(1/18) * \log(q(\text{'the'})q(\text{'president'}) \dots q(\text{'.'})) = -(1/18) * \log(q(\text{corpus}))$
 $= -(1/18) * \log(5.144e-05 * 1.240e-09) = 2.435 \text{ bit}$

This value represents the average information of each word in this sampled corpus.

(d) perplexity = $2^{H(P, Q)} = 2^{2.435}$, choose from $2^{2.435}$ words.

(e) The CFG is not able to generate the sentence "the president ate .". Therefore, $H(P, Q)$ and perplexity go to infinity and thus the sentence can't be compressed.

6.

(a) I use the command "python3 randsent.py -n 10000 -g grammar2.gr | ./parse -P -g grammar2.gr", and it automatically produces the cross-entropy result:

cross-entropy = 2.529 bits = $-(269746.803 \log\text{-prob.} / 106663 \text{ words})$

(b) grammar3 is way more complicated than grammar2, and the result says:

cross-entropy = Inf bits = $(-\text{Inf} \log\text{-prob.} / 9080 \text{ words})$

It generates extremely long sentences, which makes the probability of the sentence go to zero.

(c) grammar1 generates very long sentences which have '...' inside them. However, '...' can not be recognized by the parser and results in a zero probability. Therefore, the cross-entropy is infinity.

7. Of course it's true. The content generated by rule A should be found more surprising when we are using rule B to understand it. And this rule can be mathematically proven by using the logarithm inequality.

4.1 Questions

I chose phenomena (a) and (b), and I made following modifications to my grammar file:
phenomena (a):

I divided the scenarios into 2 clear branches:

1 NP Det Noun

1 NP Det_vowel Compound_vowel

And for Compound_vowel:

1 Compound_vowel Adj_vowel Suffixcompound_vowel

1 Suffixcompound_vowel Adj_vowel Suffixcompound_vowel

1 Suffixcompound_vowel Adj Suffixcompound_vowel

1 Suffixcompound_vowel Noun

For Noun (Noun_adjed can start with a vowel):

1 Noun Adj Noun_adjed

1 Noun_adjed Adj Noun_adjed

1 Noun_adjed Adj_vowel Noun_adjed

1 Noun_adjed Noun

1 Noun_adjed Noun_vowel

Sentence generated:

(1) chef and Sally pickled **an interesting** mysterious enormous enormous interesting castle above the castle !

(2) proposal worked behind **a mysterious apple** beside this perplexed apple near the perplexed green interesting mysterious castle with every chef near a pickle above this floor beside a green mysterious perplexed apple .

phenomena (b):

1 ROOT did NP VP ?

1 ROOT will NP VP ?

1 ROOT did NP VP that S ?

1 ROOT will NP VP that S ?

Sentence generated:

(1) did an interesting mysterious enormous pickle worked ?

(2) will each chef dropped an interesting delicious interesting pickled desk that each Sally ran ?